



线程的创建 (下)

北京理工大学计算机学院
金旭亮

Thread类Run()方法的奥秘

注意Thread类实现了Runnable接口，所以，它也有Run()方法，其源码如下所示：

```
private Runnable target;

public void run() {
    if (target != null) {
        target.run();
    }
}
```



target字段在调用Thread类的构造函数时被设置。当提供了Runnable对象时，target引用它。没有提供时，target为空。所以，如果我们从Thread类中派生子类，要重写Run方法，在其中写上“干活”的代码。

挑战



以下“别扭”的代码将运行哪个Run方法？

```
Thread th = new Thread(new Runnable() {  
    public void run() {  
        System.out.println("Runnable.Run()");  
    }  
}) {  
    public void run() {  
        //super.run();  
        System.out.println("Thread.Run()");  
    }  
};  
th.start();
```

真实项目中写这种代码，是要被扁一顿的！

答案：RunWhatMethod.java

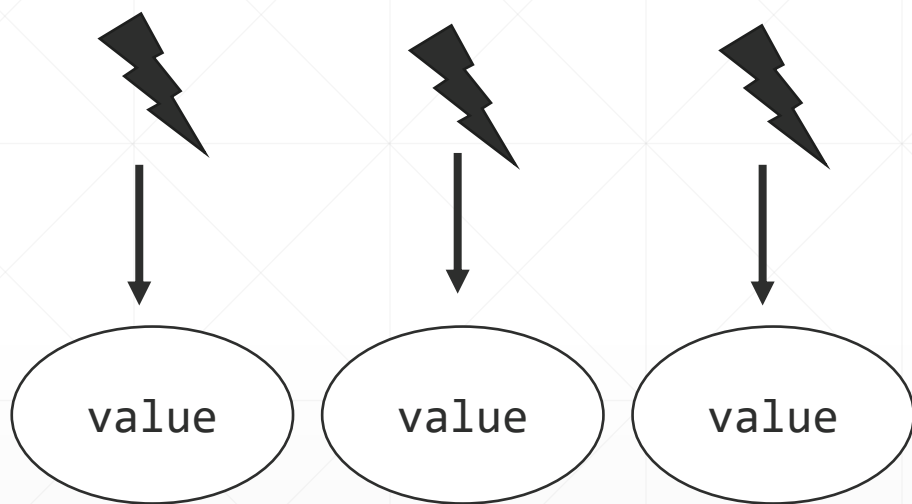
线程与线程函数

线程要干的活，由线程函数实现。线程的执行，体现为线程函数的执行。

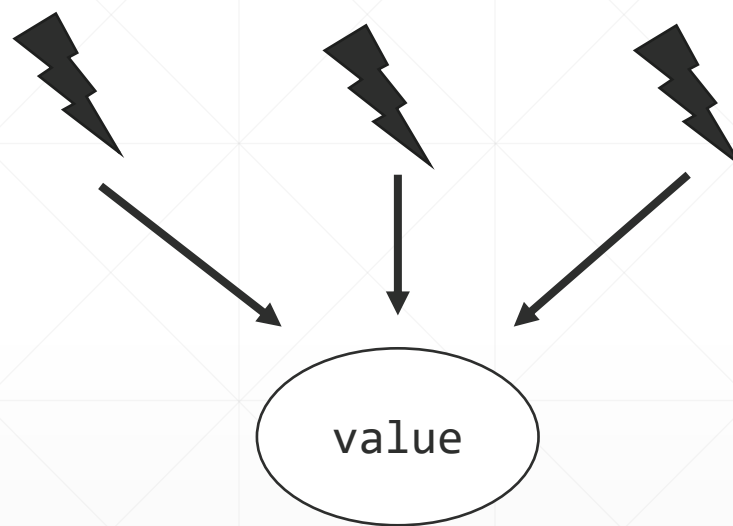


线程函数的编写，是有许多技巧的，下面我们举一个“**变量共享**”的例子.....

变量的共享与不共享



每个线程访问自己的变量



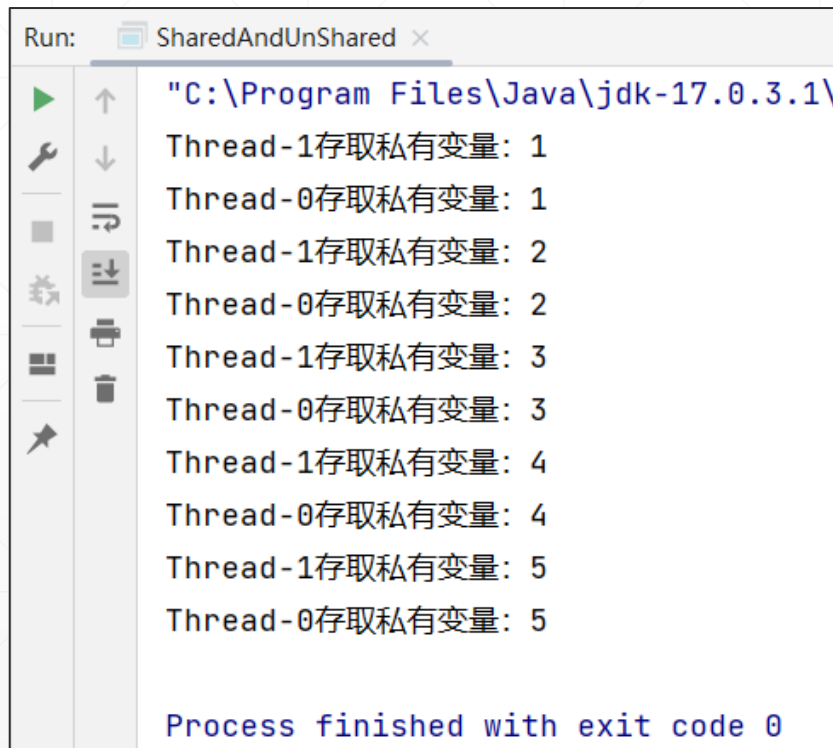
每个线程访问相同的变量

多线程访问私有变量，是安全可靠的

定义在线程函数内部的变量，是私有变量，每个线程所独享，互不影响：

```
private static void unsharedVariable() {  
    Runnable threadFunc = () -> {  
        int localvalue = 0;  
        for (int i = 0; i < 5; i++) {  
            localvalue++;  
            System.out.println(Thread.currentThread().getName()  
                + "存取私有变量: " + localvalue);  
        }  
    };  
    new Thread(threadFunc).start();  
    new Thread(threadFunc).start();  
}
```

SharedAndUnShared.java



```
Run: SharedAndUnShared x  
"C:\Program Files\Java\jdk-17.0.3.1\  
Thread-1存取私有变量: 1  
Thread-0存取私有变量: 1  
Thread-1存取私有变量: 2  
Thread-0存取私有变量: 2  
Thread-1存取私有变量: 3  
Thread-0存取私有变量: 3  
Thread-1存取私有变量: 4  
Thread-0存取私有变量: 4  
Thread-1存取私有变量: 5  
Thread-0存取私有变量: 5  
  
Process finished with exit code 0
```

多线程访问同一个外部对象

MyClass

- MyClass(int)
- getValue(): int
- setValue(int): void
- value: int = 0

```
private static void sharedObject() {  
    //在多个线程外部实例化的对象  
    MyClass shareObj = new MyClass( value: 0);  
    Runnable threadFunc = () -> {  
        for (int i = 0; i < 5; i++) {  
            //线程访问外部对象，并修改它的字段  
            shareObj.setValue(shareObj.getValue() + 1);  
            System.out.println(Thread.currentThread().getName() +  
                "存取共享对象: " + shareObj.getValue());  
        }  
    };  
    new Thread(threadFunc).start();  
    new Thread(threadFunc).start();  
}
```

SharedAndUnShared.java

运行截图



```
Run: SharedAndUnShared x
"C:\Program Files\Java\jdk-17.0.3.
Thread-1存取共享对象: 1
Thread-1存取共享对象: 3
Thread-1存取共享对象: 4
Thread-1存取共享对象: 5
Thread-1存取共享对象: 6
Thread-0存取共享对象: 2
Thread-0存取共享对象: 7
Thread-0存取共享对象: 8
Thread-0存取共享对象: 9
Thread-0存取共享对象: 10

Process finished with exit code 0
```

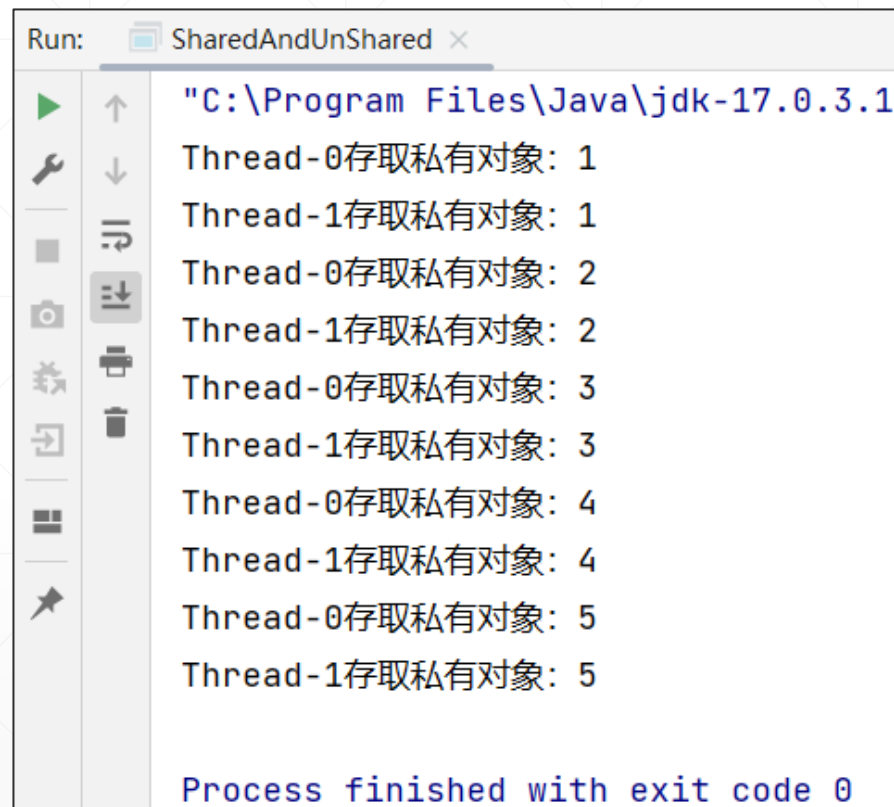
由于两个线程访问同一个对象的同一个字段，可以看到，它所存取到的值的顺序是“不唯一”和“不确定”的，并且有可能会得到不确定的结果。其原因就在于线程的运行，是由操作系统依据多种因素，采用特定的算法决定的，其推进顺序，程序员不可控。

多线程同时存取共享对象，是引发多线程数据存取错误的关键原因。

将对象“私有化”以消除不确定性

```
private static void privateObject() {  
    Runnable threadFunc = () -> {  
        MyClass privateObj = new MyClass( value: 0);  
        for (int i = 0; i < 5; i++) {  
            privateObj.setValue(privateObj.getValue() + 1);  
            System.out.println(Thread.currentThread().getName()  
                + "存取私有对象: " + privateObj.getValue());  
        }  
    };  
    new Thread(threadFunc).start();  
    new Thread(threadFunc).start();  
}
```

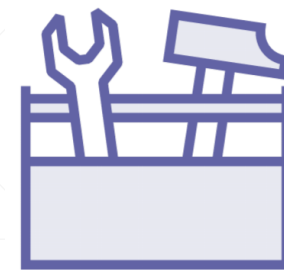
SharedAndUnShared.java



```
Run: SharedAndUnShared x  
"C:\Program Files\Java\jdk-17.0.3.1  
Thread-0存取私有对象: 1  
Thread-1存取私有对象: 1  
Thread-0存取私有对象: 2  
Thread-1存取私有对象: 2  
Thread-0存取私有对象: 3  
Thread-1存取私有对象: 3  
Thread-0存取私有对象: 4  
Thread-1存取私有对象: 4  
Thread-0存取私有对象: 5  
Thread-1存取私有对象: 5  
Process finished with exit code 0
```

将对象私有化之后，可以看到，每个线程都“独占”访问一个MyClass对象，由于不会有其他线程修改这个MyClass对象，所以，访问到的数据必然是“1->2->3->4->5”这样的顺序。

另类线程创建方式—线程工厂



JDK定义了一个ThreadFactory接口用于定义线程工厂

```
public interface ThreadFactory {  
    Thread newThread(Runnable r);  
}
```

- 实现此接口，就可以在应用程序中应用“工厂”设计模式，按需创建线程，避免了到处重复书写创建线程代码的麻烦，提升了代码的可维护性。
- 可以依据特定的需求，向外界返回特殊的自定义的Thread子类对象。
- 在需要时，可以实现一个线程池，重用而不是新建对象。
- 可以为线程对象的创建生成统计信息。

示例：ThreadFactoryDemo.java

自定义线程工厂



```
class MyThreadFactory implements ThreadFactory {  
    private int counter;  
    private final String name;  
    public MyThreadFactory(String name) {  
        counter = 0;  
        this.name = name;  
    }  
  
    @Override  
    public Thread newThread(Runnable r) {  
        Thread t = new Thread(r, name: name + "-线程-" + counter);  
        counter++;  
        System.out.printf("名字为%s的 线程%d 创建于 %s\n",  
            t.getName(), t.getId(), new Date());  
        return t;  
    }  
}
```

这个线程工厂没干啥事，就是封装传入的Runnable对象创建新线程，并设置了一下它所创建的线程的名字，而且在创建线程时输出相应的信息。

ThreadFactoryDemo.java

使用工厂创建线程



```
public class ThreadFactoryDemo {  
    public static void main(String[] args) {  
        var factory = new MyThreadFactory( name: "自定义线程工厂");  
        var scanner = new Scanner(System.in);  
        String userInput = "";  
        System.out.println("敲回车创建新线程, q退出");  
        do {  
            factory.newThread(new Runnable() {  
                @Override  
                public void run() {  
                    long threadId = Thread.currentThread().getId();  
                    System.out.println("线程" + threadId + "执行.");  
                }  
            }).start();  
            userInput = scanner.nextLine();  
        } while (!Objects.equals(userInput, b: "q"));  
        System.out.println("演示结束");  
    }  
}
```

ThreadFactoryDemo.java

```
Run: ThreadFactoryDemo x  
"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe" "-javaagent:C:\Program  
敲回车创建新线程, q退出  
名字为"自定义线程工厂-线程-0"的 线程24 创建于 Thu Aug 18 21:50:02 CST 2022  
线程24执行。  
  
名字为"自定义线程工厂-线程-1"的 线程25 创建于 Thu Aug 18 21:50:11 CST 2022  
线程25执行。  
  
q  
演示结束  
  
Process finished with exit code 0
```

巩固练习



主线程启动一个工作线程，这个工作线程负责找出 $[1, 1000]$ 区间内所有的素数。



主线程启动一个工作线程，这个工作线程负责找出 $[1, 1000]$ 区间内所有的素数，这个工作线程又启动了两个线程，一个负责 $[1, 500)$ 这个区间，另一个负责 $[500, 1000]$ 这个区间。